

CardiolA

A Nova Era da Cardiologia Inteligente

IR ALÉM 1 — PORTAL CLÍNICO

React + Vite com Context API

Documento Resumo da Entrega

FIAP — Graduação em Inteligência Artificial

Grupo Purkinje · Turma 2TIAOR-2026

Integrante	RM	Turma
Miriã Leal Mantovani	RM567811	2TIAOR-2026
João Pedro Santos Azevedo	RM566701	2TIAOR-2026

Tutor: Leonardo Ruiz Orabona

LINKS DA ENTREGA

Repositório	https://github.com/joastazevedo172/purkinje-cardioia-portal
Vídeo (YouTube)	https://youtu.be/GM7ARIVafeE
Projeto principal	https://github.com/joastazevedo172/cardioia-fase2

Setembro de 2026

SUMÁRIO

1 VISÃO GERAL DA ENTREGA.....	3
1.1 O que foi construído.....	3
1.2 Entregáveis exigidos.....	3
2 ARQUITETURA DA APLICAÇÃO.....	5
2.1 Por que dois contextos.....	5
2.2 Organização das pastas.....	5
3 FUNCIONALIDADES.....	7
3.1 Autenticação e sessão.....	7
3.2 Proteção de rotas.....	7
3.3 Listagem de pacientes.....	8
3.4 Agendamento de consultas.....	8
3.5 Dashboard.....	8
4 DECISÕES TÉCNICAS.....	10
4.1 useReducer no lugar de vários useState.....	10
4.2 useMemo nos valores de contexto.....	10
4.3 AbortController nos efeitos de busca.....	10
4.4 Estilização com CSS Modules.....	11
5 RESPONSABILIDADE E ACESSIBILIDADE.....	12
6 EXECUÇÃO E LIMITAÇÕES.....	12

1 VISÃO GERAL DA ENTREGA

Esta atividade pede a construção da interface do Cardiola: um portal responsivo que simule a rotina de um serviço de diagnóstico em cardiologia. Não há back-end, e os dados vêm de uma API pública de exemplo. O foco recai sobre componentização, navegação, autenticação simulada e manipulação de estado com Hooks.

1.1 O que foi construído

O portal tem quatro telas. A de login autentica contra uma lista local de usuários e grava a sessão no navegador. O dashboard reúne indicadores de pacientes e consultas. A listagem de pacientes consome uma API pública e oferece busca e filtro. O agendamento traz um formulário validado ao lado da agenda, que se atualiza em tempo real.

Nenhuma biblioteca de interface, de formulário ou de gerenciamento de estado externa foi utilizada. Tudo o que aparece na tela foi escrito pelo grupo, com React, React Router e CSS Modules — decisão que nos obrigou a entender os mecanismos em vez de delegá-los.

1.2 Entregáveis exigidos

Requisito do enunciado	Implementação	Onde
Autenticação simulada via Context API com JWT falso no localStorage	AuthContext com useReducer, token em base64url e validade de 8 horas	contexts/AuthContext.jsx
Listagem de pacientes usando API fake	Consumo do JSONPlaceholder com enriquecimento clínico determinístico	services/pacientesService.js
Formulário de agendamento com useState e useReducer	Formulário com reducer para validação e useState para a lista carregada	pages/Agendamento.jsx
Dashboard com contagem de pacientes e consultas	Quatro indicadores, distribuição por risco e próximas consultas	pages/Dashboard.jsx
Proteção de rotas com AuthContext	Componente RotaProtegida com três estados	components/RotaProtegida.jsx
Estilização com CSS Modules ou	CSS Modules, um arquivo por	styles/

Requisito do enunciado	Implementação	Onde
Styled Components	componente, com tokens em variáveis CSS	

2 ARQUITETURA DA APLICAÇÃO

A aplicação é organizada em duas camadas de contexto que envolvem todas as rotas. A camada externa cuida da sessão; a interna, da agenda de consultas. Dentro delas ficam as rotas: uma pública, a de login, e três protegidas.

2.1 Por que dois contextos

A separação não é estética: os dois estados têm ciclos de vida diferentes. A sessão nasce no login, é gravada no armazenamento do navegador e sobrevive a um recarregamento de página. A agenda de consultas vive apenas na memória da aplicação e se perde a cada recarregamento, porque não há servidor que a persista.

Unificar os dois em um contexto único traria um efeito indesejado: cada consulta agendada dispararia uma nova renderização em todos os componentes que consomem a sessão — o cabeçalho, a guarda de rota — sem que nada relacionado à sessão tivesse mudado.

Contexto	Estado que guarda	Quem lê	Quem escreve
AuthContext	usuário, token, carregando, erro	Header, RotaProtegida, Dashboard, Login	Login, Header
ConsultasContext	lista de consultas e métricas derivadas	Dashboard, Agendamento	Agendamento

2.2 Organização das pastas

A estrutura segue exatamente o que o enunciado pede, com uma responsabilidade clara por pasta:

Pasta	Responsabilidade
contexts/	Estado global da aplicação. Dois provedores: sessão e agenda.
components/	Peças reutilizáveis e sem regra de negócio: guarda de rota, cabeçalho, cartão de indicador e etiqueta de status.
services/	Tudo que conversa com o mundo externo: a API de pacientes e a geração e

Pasta	Responsabilidade
	leitura do token.
pages/	Uma tela por arquivo, montada a partir dos componentes e dos contextos.
styles/	Um módulo CSS por componente, mais um arquivo global com os tokens de design.

O isolamento da pasta de serviços tem uma consequência prática: trocar o JSONPlaceholder por um back-end real exigiria alterar um único arquivo, sem tocar em nenhum componente.

3 FUNCIONALIDADES

3.1 Autenticação e sessão

O login valida as credenciais contra uma lista local de dois usuários, com papéis distintos: cardiologista e administrador. A chamada tem atraso simulado de 600 milissegundos, tempo suficiente para que os estados de carregamento apareçam de verdade durante uma demonstração — sem ele, a transição seria instantânea e o tratamento de estado ficaria invisível.

Em caso de sucesso, o serviço gera um token no formato JWT e o grava no armazenamento local junto com a data de expiração, definida em oito horas. Ao montar, o provedor lê esse registro e restaura a sessão; se a validade passou, o registro é descartado.

Sobre o token: dizemos em voz alta que ele é falso

O token tem a estrutura correta de um JWT — cabeçalho, carga e assinatura em base64url — e a carga traz o e-mail, o papel e a expiração. Mas a assinatura é uma string fixa. Qualquer servidor real o rejeitaria imediatamente. Isso é proposital, e o portal declara essa limitação em dois lugares da interface: na tela de login e no painel que decodifica o token. Segurança de autenticação só existe quando o servidor assina e valida; o que o front-end pode fazer é gerenciar a sessão, e é exatamente isso que está implementado aqui.

3.2 Proteção de rotas

Um componente chamado RotaProtegida envolve as três rotas privadas e consulta o contexto de sessão antes de renderizar qualquer coisa. Se não houver sessão, redireciona para o login guardando o caminho de origem — assim, depois de autenticar, o usuário chega ao destino que tentou abrir, e não à página inicial.

Esse componente tem três estados, e não dois. O terceiro existe por um motivo concreto: a leitura do armazenamento local acontece dentro de um efeito, ou seja, depois do primeiro render. Sem um estado explícito de "verificando sessão", um usuário já

autenticado seria redirecionado ao login a cada recarregamento de página, no intervalo entre montar o provedor e restaurar a sessão.

3.3 Listagem de pacientes

A lista é carregada do JSONPlaceholder, a API pública sugerida no enunciado. Como ela devolve usuários genéricos, sem qualquer informação clínica, cada registro é enriquecido com idade, condição, pressão arterial, frequência cardíaca e nível de risco.

Esse enriquecimento é determinístico: os valores derivam do identificador do usuário por meio de operações aritméticas fixas. A consequência é que o mesmo paciente sempre exibe os mesmos dados, em qualquer recarregamento — algo que valores aleatórios não garantiriam, e que tornaria a demonstração confusa.

A tela oferece busca por nome, e-mail ou cidade e filtro por nível de risco, ambos aplicados sobre a lista já carregada. Se a API estiver indisponível, o serviço recorre a uma base local de seis pacientes, de modo que a demonstração nunca falhe por ausência de conexão.

3.4 Agendamento de consultas

O formulário reúne paciente, data, horário, tipo de exame, prioridade e observação clínica. A validação bloqueia o envio sem paciente, data ou horário, recusa datas no passado e limita a observação a duzentos caracteres, com contador visível.

Ao lado do formulário fica a agenda, que se atualiza imediatamente a cada novo agendamento porque ambos leem do mesmo contexto. Cada consulta pode ser concluída ou cancelada, e o dashboard reflete essas mudanças sem qualquer sincronização manual.

3.5 Dashboard

A tela inicial reúne quatro indicadores: pacientes cadastrados, consultas agendadas, consultas de prioridade alta e consultas realizadas. Abaixo deles, a distribuição dos pacientes por risco cardiovascular em barras proporcionais e as quatro próximas consultas da agenda, ordenadas por data e horário.

O dashboard é o único ponto da aplicação que consome os dois contextos ao mesmo tempo, o que o torna uma boa demonstração da separação entre eles. Traz ainda um painel expansível que decodifica e exibe a carga do token da sessão.

4 DECISÕES TÉCNICAS

4.1 `useReducer` no lugar de vários `useState`

O `reducer` aparece em dois lugares do projeto, e nos dois pelo mesmo motivo: as transições de estado envolvem mais de um campo simultaneamente.

No contexto de autenticação, iniciar a verificação precisa limpar o erro anterior; concluir com sucesso precisa preencher usuário e token e zerar o erro; encerrar a sessão precisa limpar tudo. Com quatro variáveis de estado independentes, cada transição exigiria quatro chamadas coordenadas, e esquecer uma delas deixaria, por exemplo, uma mensagem de erro antiga visível durante uma nova tentativa.

No formulário de agendamento, alterar um campo precisa apagar o erro daquele campo específico, enviar precisa validar todos os campos em conjunto, e limpar precisa restaurar o formulário inteiro. O `reducer` concentra essas regras em um único ponto, onde ficam legíveis e testáveis.

4.2 `useMemo` nos valores de contexto

O objeto entregue por cada provedor é memorizado. Sem isso, ele seria recriado a cada renderização do provedor, e o React trataria essa nova referência como mudança — fazendo todos os componentes consumidores renderizarem de novo, mesmo sem nenhuma alteração real de dado. É um erro comum e silencioso: a aplicação continua correta, apenas desperdiça trabalho.

4.3 `AbortController` nos efeitos de busca

Toda requisição feita dentro de um efeito recebe um sinal de cancelamento, e a função de limpeza do efeito o aciona. Se o usuário navegar para outra página antes de a requisição terminar, ela é abortada.

Sem esse cuidado, o React emite aviso sobre atualização de estado em componente já desmontado. No modo estrito, que monta cada componente duas vezes

durante o desenvolvimento, o aviso apareceria sempre — e o desenvolvedor acabaria se acostumando a ignorá-lo, que é como avisos úteis se perdem.

4.4 Estilização com CSS Modules

Optamos por CSS Modules em vez de Styled Components por duas razões. A primeira é que não adiciona nenhuma dependência ao pacote final: o escopo por arquivo é resolvido em tempo de compilação pelo próprio Vite. A segunda é que mantém o CSS como CSS, o que facilita a leitura para quem revisa o código.

Os tokens de design — cores, raio de borda, sombra, família tipográfica — ficam declarados como variáveis CSS em um arquivo global, servindo de ponto único de verdade. Trocar a cor de destaque do portal inteiro exige alterar uma linha.

5 RESPONSABILIDADE E ACESSIBILIDADE

A responsividade foi obtida apenas com CSS, sem framework. Três mecanismos sustentam o comportamento em telas menores:

- Grades fluidas com repeat e minmax: os cartões do dashboard se reorganizam sozinhos conforme a largura disponível, sem nenhum ponto de quebra declarado.
- Abaixo de 900 pixels, a tela de agendamento passa de duas colunas para uma, e o formulário deixa de dividir data e horário lado a lado.
- Abaixo de 720 pixels, a navegação quebra para uma linha própria e a identificação do usuário é ocultada, preservando o botão de sair.

A tabela de pacientes recebe rolagem horizontal em vez de espremer as colunas, decisão deliberada: dados clínicos em coluna estreita demais ficam ilegíveis, e rolar é preferível a adivinhar.

Do lado da acessibilidade, os campos de busca e filtro têm rótulo acessível, as mensagens de erro são anunciadas como alerta e as de carregamento como status. O foco é visível na navegação por teclado, e a animação do indicador de carregamento é desligada para quem configurou preferência por movimento reduzido. Todas as combinações de texto e fundo mantêm contraste acima de 4,5 para 1.

6 EXECUÇÃO E LIMITAÇÕES

6.1 Como executar

O projeto exige Node.js 18 ou superior. Após clonar o repositório, bastam dois comandos: instalar as dependências e iniciar o servidor de desenvolvimento, que abre o navegador automaticamente. A compilação para produção foi verificada e conclui em cerca de três segundos, gerando 189 kilobytes de JavaScript e 12 de CSS.

A tela de login traz botões que preenchem as credenciais de demonstração automaticamente, de modo que não é necessário digitar nada para acessar o portal.

6.2 Limitações declaradas

- A autenticação não protege nada. É simulada por definição: qualquer pessoa com o console do navegador pode gravar um token no armazenamento local e obter acesso.
- Não há back-end. As consultas agendadas vivem apenas na memória da aplicação e desaparecem ao recarregar a página.
- Os dados clínicos dos pacientes são derivados do identificador. A API pública devolve usuários genéricos, sem informação médica.
- Não há testes automatizados. O escopo da atividade é a interface; uma suíte com Vitest e Testing Library seria o próximo passo natural.

6.3 Referências

- REACT. Documentação oficial: Hooks e Context. Disponível em react.dev.
- REACT ROUTER. Documentação da versão 6: navegação e rotas aninhadas.
- JONES, M.; BRADLEY, J.; SAKIMURA, N. RFC 7519 — JSON Web Token (JWT). IETF, 2015.
- VITE. Guia de build e configuração. Disponível em vitejs.dev.

Aviso

Todos os dados exibidos no portal são simulados. A autenticação é fictícia e não oferece segurança real. Nenhum conteúdo constitui orientação médica. O projeto destina-se exclusivamente ao desenvolvimento acadêmico.